*Chapitre 1 :*

# Introduction au Web

Ce premier chapitre pose les bases indispensables à tout développeur web : il clarifie les notions fondamentales d'internet et du web, décortique les mécanismes de fonctionnement du web, présente un aperçu des technologies de base ainsi que le cycle de vie d'un projet Web.

## I. Internet vs Web

Avant d'étudier la création de pages Web, il est essentiel de lever une confusion fréquente en distinguant le réseau physique mondial du service applicatif qu'est le Web.

### I.1. L'internet

L'internet : **Inter**connected **Net**works est un réseau mondial de réseaux informatiques interconnectés qui communiquent entre eux grâce à un ensemble de protocoles standardisés appelés TCP/IP (Transport Control Protocol/Internet Protocol).

Internet fournit aujourd'hui une multitude de services à travers le monde tels que le courrier électronique, le transfert de fichiers, le world wide web, la voix sur IP, …

### I.2. Le World Wide Web

Le World Wide Web (littéralement la "Toile mondiale", abrégé WWW ou Web) est un système d'information décentralisé fonctionnant sur Internet inventé en 1989 par Tim Berners-Lee.

Il permet de consulter des ressources (pages, images, vidéos, …) reliées entre elles par des liens hypertextes, identifiées par des adresses uniques (URL) et accessibles via un navigateur grâce au protocole HTTP.

## II. Fonctionnement du Web

Le Web s'appuie sur une architecture **Client/Serveur** dans laquelle les échanges sont régis par le protocole **HTTP** (HyperText Transfer Protocol).

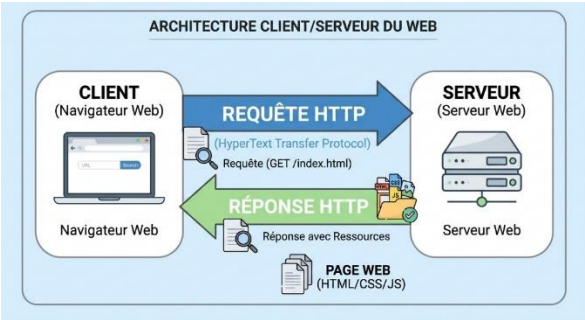



**Figure 1: Modèle Client / Serveur**

## II.1. Les composants clés de l'écosystème web

Les acteurs principaux lors d'un échange sur le web sont :

- Le **Navigateur Web** (Client) : Logiciel chargé d'envoyer des requêtes, puis d'analyser le code reçu (HTML, CSS, JavaScript) pour générer un rendu visuel interprétable par l'utilisateur.

*Exemples* : Google Chrome, Opera, Mozilla Firefox, Apple Safari, Microsoft Edge,…

- Le **Serveur Web** (Serveur) : Logiciel hébergé sur une machine distante qui écoute les requêtes des clients, traite les données et renvoie les ressources demandées

*Exemples :* Apache, Nginx, Microsoft IIS, …

- La **Page Web** : Document numérique structuré (principalement en HTML/CSS), identifié par une adresse unique (URL) et pouvant contenir du texte, du multimédia ou des éléments interactifs.

- Le Protocole **HTTP (HyperText Transfer Protocol) / HTTPS :** Ensemble de règles standardisées permettant le transfert des données (texte, images, code) entre le client et le serveur. Il définit des méthodes d'action (comme GET pour récupérer une page, ou POST pour envoyer un formulaire).

## II.2. Structure d'une URL (Uniform Resource Locator)

Chaque ressource du Web est localisée grâce à une URL (*Uniform Resource Locator*).

*Exemples :* https://www.google.com
https://iset.uvt.tn/course/view.php?id=21
https://www.weblearn.tn/tutoJS/TP2/index.html

**Tableau 1: Eléments constitutifs d'une URL**

| Composant | Rôle | Exemples |
|---|---|---|
| Protocole | Définit la méthode de communication | https<br>http |
| Hôte (Sous domaine) | Identifie le serveur sur le réseau | www,<br>iset |
| Domaine de deuxième niveau (SLD : Second Level Domain) | nom propre qu'une organisation, une entreprise ou un particulier enregistre auprès d'un registrar pour s'identifier. | Google<br>Uvt<br>weblearn |
| Domaine de premier niveau (TLD : Top Level Domain) | C'est l'extension, il peut être générique ou national | gTLD : .com, .org<br>ccTLD : .tn, .fr, .ca |
| Nom de domaine | SLD.TLD | google.com<br>uvt.tn<br>weblearn.tn |
| Chemin d'accès | Emplacement du dossier sur le serveur | /course/<br>/tutoJS/TP2/ |

| | | |
|---|---|---|
| Ressource / Fichier | Nom de la ressource demandée | view.php<br>index.html |
| Chaîne de requête | Chaîne située après le ? qui définit les données à transmettre au serveur | id=21 |

### II.3. Le cycle de fonctionnement : De la requête à l'affichage

Depuis la saisie d'une URL dans le navigateur jusqu'à l'affichage complet de la page web, plusieurs étapes successives interviennent. Elles sont synthétisées dans le schéma suivant :

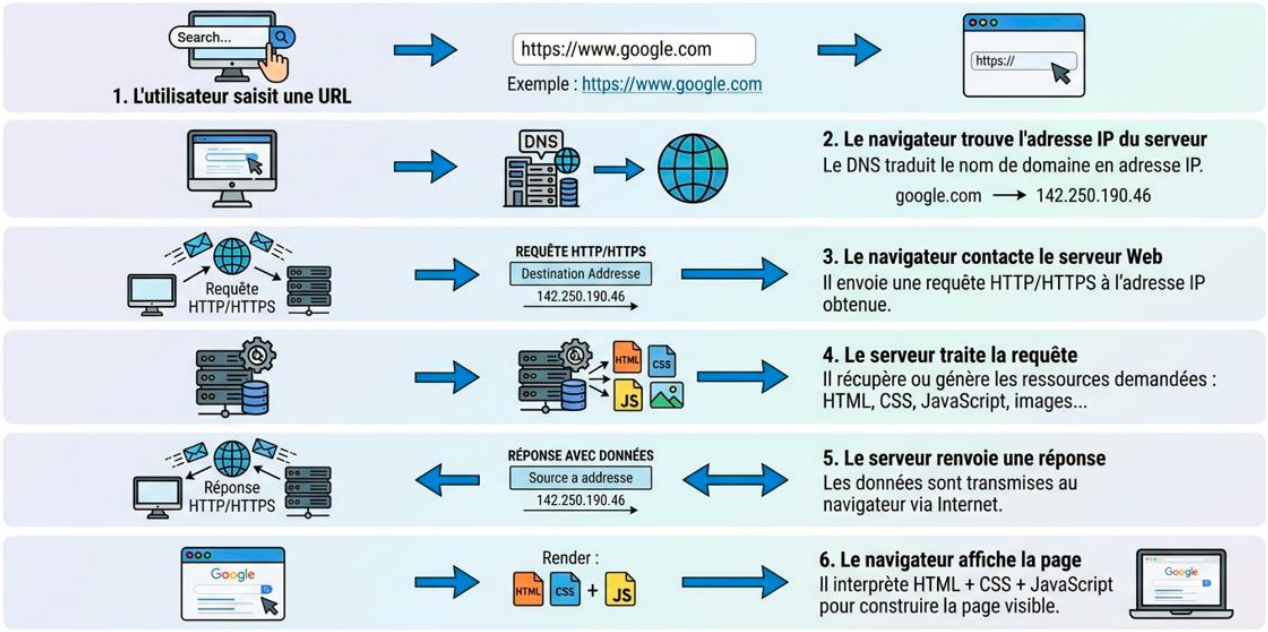



**Figure 2: Cycle de fonctionnement du web**

## III. Technologies du Web

L'affichage d'un site Web est le résultat d'une collaboration étroite entre le code exécuté dans le navigateur et les traitements réalisés sur le serveur.

### III.1. Frontend vs Backend (Côté Client / Côté Serveur)

Le développement Web se divise en deux grands domaines complémentaires :

- **Le Frontend (Côté Client) :** Désigne tout ce qui est exécuté directement par le navigateur Web de l'utilisateur. Il regroupe l'interface visuelle, la structure des documents, la mise en forme graphique et les interactions utilisateur directes.

- **Le Backend (Côté Serveur) :** Désigne la partie "invisible" hébergée sur le serveur Web. Il prend en charge la logique métier de l'application, l'authentification des utilisateurs, le traitement des formulaires et la communication avec les bases de données (ex: MySQL, PostgreSQL).

### III.2. Les langages et outils du FrontEnd

Pour construire une interface Web moderne, le navigateur combine trois langages fondamentaux, souvent accompagnés de frameworks de mise en forme :

- **HTML (H**yper**T**ext **M**arkup **L**anguage**) :** Le langage de balisage qui définit la structure et le contenu de la page Web (titres, paragraphes, images, liens, formulaires).
- **CSS (C**ascading **S**tyle **S**heets**) :** Le langage de style qui gère la présentation visuelle et le design (couleurs, typographies, disposition des éléments, réactivité sur mobile).
- **JavaScript (JS) :** Le langage de programmation qui apporte le dynamisme et l'interactivité côté client (animations, validation de formulaires en direct, mise à jour du contenu sans recharger la page, accès aux données serveur,…).

Afin d'accélérer la création d'interfaces, les développeurs s'appuient souvent sur des outils clés en main :

- *Les frameworks CSS* (ex: Bootstrap, Tailwind CSS, daisyUI,…) : pour concevoir rapidement des interfaces modernes et réactives.
- *Les bibliothèques JavaScript* (ex: jQuery, Chart.js, Anime.js,…) : pour simplifier l'écriture des scripts et les interactions avec la page Web.
- *Des frameworks JS complets* (ex: Angular, Next.js, Vue,…) : pour une gestion complète de l'application.

## IV. Workflow du développement Web

Le développement d'un projet Web ne se limite pas à l'écriture du code ; il suit un cycle de vie structuré qui va de la conception initiale jusqu'à la mise en ligne en passant par le versionnement collaboratif.

**1. Phase de Conception & Maquettage**

- Définition des besoins et structuration de l'arborescence du site.
- Création de maquettes fonctionnelles (wireframes) et visuelles (UI/UX) pour valider l'ergonomie avant le codage.

**2. Phase de Gestion de Version & Collaboration (Git & GitHub) :**

- Contrôle de version (Git) : Suivi de l'historique des modifications du code source, gestion des branches de travail et possibilité de revenir à des versions antérieures.
- Plateformes d'hébergement de code (ex: GitHub, GitLab, Bitbucket) : Centralisation du dépôt (repository), facilitation du travail en équipe, revue de code et automatisation des déploiements.

**3. Phase de Développement (Codage)**

- Intégration Frontend : Traduction des maquettes en code HTML/CSS et ajout d'interactivité en JavaScript.

- Développement Backend : Conception de la base de données, création des API et écriture de la logique métier côté serveur.

**4. Phase de Test & Validation**

- Vérification de l'affichage sur différents navigateurs (Cross-browser compatibility) et appareils (Responsive Design).
- Validation des fonctionnalités, des formulaires et contrôle de la sécurité applicative.

**5. Phase de Déploiement (Mise en ligne) :**

- Réservation du nom de domaine auprès d'un registrar.
- Configuration de l'hébergement et transfert du projet vers le serveur de production (via des services de déploiement continu liés à GitHub/GitLab ou par protocole sécurisé SFTP).

En résumé, afficher une page web est le résultat de tout un enchaînement d'actions : requête http, traitement serveur et rendu client. La maîtrise de ces principes constitue le socle indispensable pour passer à la pratique avec le langage de structuration : le HTML.